

Global vs. local state

COMPLETE LEARNING GUIDE

What You'll Learn

Every frontend framework eventually forces the same architectural question: should a given piece of state live inside the one component that needs it, or somewhere shared that many components can read and change? This submodule addresses that question at the conceptual level, deliberately independent of any one framework's specific API, since the underlying judgment (not the syntax) is what actually transfers between React, Angular, and Vue jobs. It covers the signs that state should stay local, the signs it should be lifted or centralized, the tradeoffs global state introduces (debuggability, re-render cost, testability), the idea of derived state versus duplicated state, and a brief survey of the global-state tools available in each of the three major frameworks so the concept maps cleanly onto whichever one a given project uses.

1. Defining Local vs. Global State

Local state is owned by, and relevant only to, a single component — a dropdown's open/closed flag, the current value of a text input before it's submitted, a hover state for a tooltip. Global (or shared/app-level) state is needed by multiple components that aren't in a direct parent-child relationship, or that are too distant in the tree for passing props down at every level to remain practical — the identity of the logged-in user, a shopping cart's contents, the current UI theme, notification/toast messages. The distinction isn't about how 'important' the data is; it's purely about how many independent parts of the UI need to read or change it.

2. Signs State Should Stay Local

If no other component needs to know about a piece of state, and it would be lost with no consequence if the component unmounted and remounted, it almost always belongs as local state in that component, managed with whatever your framework's local state primitive is (useState in React, a plain component property in Angular/Vue with the Composition API's ref()). Keeping state local by default — resisting the urge to centralize 'just in case a component needs it later' — keeps components genuinely independent, easier to test in isolation, and easier to delete or move without hunting down every place that reads their state from a shared store.

3. Signs State Should Be Lifted or Made Global

State should move upward, and potentially all the way to a global store, once a second, unrelated part of the tree genuinely needs to read or change the same value — a filter selection that must affect both a sidebar and a results grid that don't share a close parent, a logged-in user's identity needed on nearly every screen, a cart badge count shown in a header far from the product-listing components that add items to it. The test isn't 'might this be useful elsewhere someday' (which almost anything could satisfy) but 'is there a second component, right now, that needs this.'

4. Lifting State Up as the Intermediate Step

Before reaching for a global store, lifting state to the nearest common ancestor of the components that need it is usually the right intermediate step — this works identically in concept across React (moving a `useState` call up and passing props/callbacks down), Angular (moving state to a shared parent component or a scoped service), and Vue (moving a `ref` up and passing it via props/emits, or a `provide/inject` pair). Only once state needs to be shared across parts of the tree with no reasonably close common ancestor — or across entirely separate route-level pages — does a genuinely global store become the more practical solution than continuing to lift state through increasingly distant ancestors.

5. Global State Tools Across Frameworks

Each major framework has both a lightweight, built-in option and heavier, more structured dedicated libraries for global state. React offers the built-in Context API for simpler cases and libraries like Redux Toolkit or Zustand for larger, more complex shared state. Angular commonly uses injectable services (which are singletons by default within their provided scope) combined with RxJS Observables or, increasingly, Signals, to share and reactively propagate state. Vue offers a built-in `reactive()` store pattern for simple cases and Pinia (the current officially recommended state library, having succeeded Vuex) for larger applications. Despite the different names and APIs, all of them solve the identical problem: a single source of truth that multiple, otherwise-unrelated components can subscribe to and update.

6. Derived State: Don't Duplicate What You Can Compute

A subtle but common state-management mistake is storing a value in state that could instead be calculated on the fly from other state you already have — a 'filtered items' array stored separately from the full items array and the current filter text, for example. Storing both invites the two to drift out of sync (you filter once, then forget to re-filter when the underlying items array changes elsewhere), whereas computing the filtered list directly during render from the two source values (or memoizing that computation) guarantees it's always consistent with its inputs by construction, with no separate update step that can be forgotten.

7. The Real Tradeoffs of Global State

Global state isn't free — it makes an application harder to reason about, since any given piece of shared state could in principle be read or changed from anywhere in the codebase, which becomes a genuine debugging burden past a certain scale ('what changed this value, and from where?'). It also makes components less portable and harder to unit test in isolation, since components depending on global state typically need that global context set up (or mocked) just to render for a test. The practical implication: treat global state as a deliberate architectural decision made when local/lifted state genuinely can't solve the problem — not a default reached for out of convenience or to avoid deciding where state should actually live.

Common Pitfalls

- Defaulting new state to a global store 'just in case it's needed elsewhere,' when no second component currently needs it.
- Storing a value in state that could instead be derived/computed from other existing state, letting the two drift out of sync.
- Lifting state all the way to a global store when lifting it only as far as the nearest common ancestor would have been sufficient.
- Treating a global store as a dumping ground for anything shared by even two components, rather than reserving it for genuinely cross-cutting, app-wide concerns.
- Making components hard to test or reuse because they reach directly into a global store instead of receiving data through their normal props/inputs.
- Forgetting that syncing two independent sources of truth for the 'same' piece of data (e.g. a local copy of server data plus the server itself) is itself a state-duplication bug waiting to happen.

Key Takeaways

- The local-vs-global decision is about how many independent parts of the UI need the data right now, not how important the data feels.
- Lift state to the nearest common ancestor before reaching for a global store — most 'shared state' problems don't need a full store.
- This concept transfers directly across React, Angular, and Vue — only the specific tool names (Context/Redux, services/RxJS, Pinia/provide-inject) differ.
- Prefer deriving state from existing data over storing a separate, parallel copy that can drift out of sync.
- Global state has real costs — harder debugging, harder isolated testing — so treat it as a deliberate choice, not a default.

Learn More

React — Sharing State Between Components

<https://react.dev/learn/sharing-state-between-components>

Pinia — Official Documentation (Vue)

<https://pinia.vuejs.org/>

Kent C. Dodds — State Colocation will make your React app faster

<https://kentcdodds.com/blog/state-colocation-will-make-your-react-app-faster>